
Sistemas Operacionais

Interação entre Tarefas - Parte II

Rogério Pereira Junior

Instituto Federal de Santa Catarina
campus São José
rogeriopereirajunior@gmail.com

28 de abril de 2026



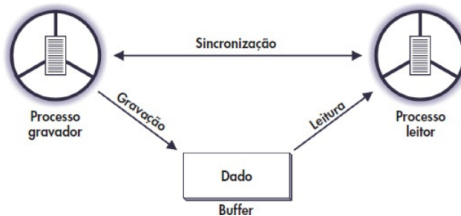
- Os processos são isolados pelo Sistema Operacional para segurança.
- **IPC (Inter-Process Communication):** Conjunto de mecanismos que permitem que processos troquem dados.
- **Principais mecanismos:**
 - 1 Pipes (Tubulações)
 - 2 Filas de Mensagens (Message Queues)
 - 3 Memória Compartilhada (Shared Memory)



Coordenação entre tarefas



- Muitas implementações de sistemas complexos são estruturadas como várias tarefas interdependentes
 - Cooperam entre si
 - Necessidade de comunicação para conseguir a cooperação
- Problemas podem ocorrer nesse processo de comunicações
- Veremos esses problemas e algumas possíveis soluções (simples em primeiro momento).



Quais problemas podem ocorrer quando 2 ou mais tarefas acessam de maneira concorrente os mesmos recursos?



Concorrência

- Capacidade de lidar com múltiplas tarefas ao mesmo tempo.
- Não significa execução simultânea, mas alternância entre tarefas.
- Exemplo: multitarefa em um processador de único núcleo.

Paralelismo

- Execução real e simultânea de múltiplas tarefas.
- Requer múltiplos núcleos ou processadores.
- Exemplo: processamento paralelo em um computador com vários núcleos.



Aspecto	Concorrência	Paralelismo
Definição	Alternância entre tarefas	Execução simultânea
Foco	Coordenação	Performance
Hardware	Pode ocorrer em 1 núcleo	Precisa de múltiplos núcleos
Exemplo	Multitarefa em PC comum	Supercomputadores, CPUs multi-core



O problema da concorrência

Quando duas ou mais tarefas acessam simultaneamente um recurso compartilhado, podem ocorrer problemas de consistência dos dados ou do estado do recurso acessado.

Exemplo de Condição de Corrida



Exemplo: Depósito Bancário

Considere uma função simples de depósito:

Código C

```
void depositar (long conta, long valor) {  
    (*saldo) += valor;  
}
```

- Em nível de montagem (Assembly), essa linha se torna várias instruções: carregar memória, somar no registrador e salvar de volta.
- **Analogia:** Duas pessoas tentando escrever em um post-it ao mesmo tempo. Se ambas leem "0", e uma soma 50 e a outra 1000, o resultado final dependerá de quem "colar" o post-it por último.



Exemplo: Depósito Bancário

- gcc -Wall -c -O0 depositar.c
- objdump -no-show-raw-insn -d depositar.o

```
0000000000000000 <depositar>:
    ; carrega o conteúdo da memória apontada por "saldo" em EDX
    mov     -0x8(%rbp),%rax      ; saldo → rax (endereço do saldo)
    mov     (%rax),%edx         ; mem[rax] → edx

    ; carrega o conteúdo de "valor" no registrador EAX
    mov     -0xc(%rbp),%eax     ; valor → eax

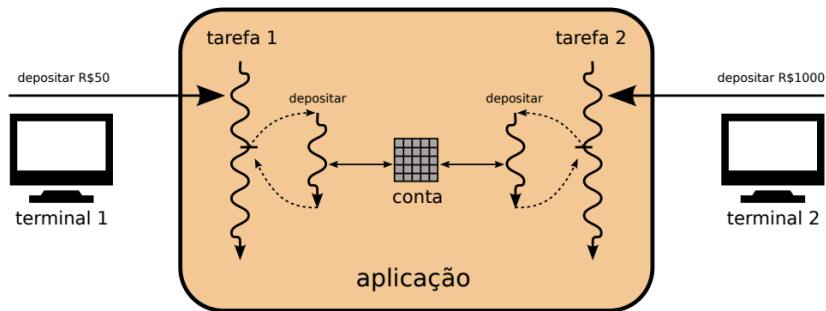
    ; soma EAX ao valor em EDX
    add     %eax,%edx           ; eax + edx → edx

    ; escreve o resultado em EDX na memória apontada por "saldo"
    mov     -0x8(%rbp),%rax     ; saldo → rax
    mov     %edx,(%rax)         ; edx → mem[rax]
```

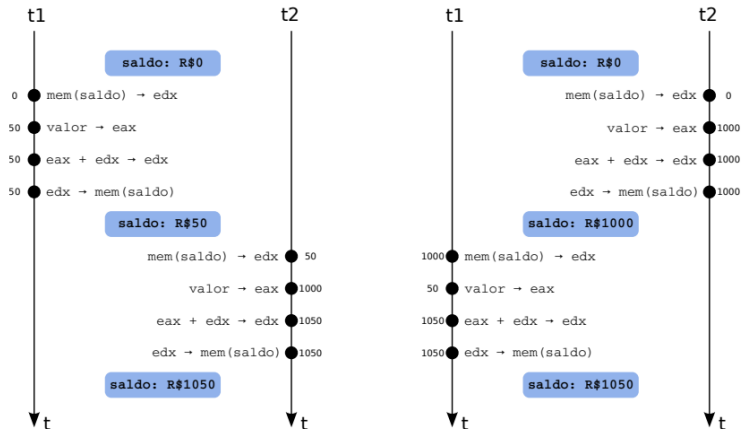
- As operações aritméticas ocorrem nos registradores
 - O valor da variável saldo (que está na memória) é copiado para um registrador da CPU (um espaço interno ultrarrápido).
 - a soma **temp** += **valor** não é feita diretamente na memória, mas dentro do registrador. O processador pega o valor de **temp** e **valor**, coloca-os em registradores e executa a soma ali.



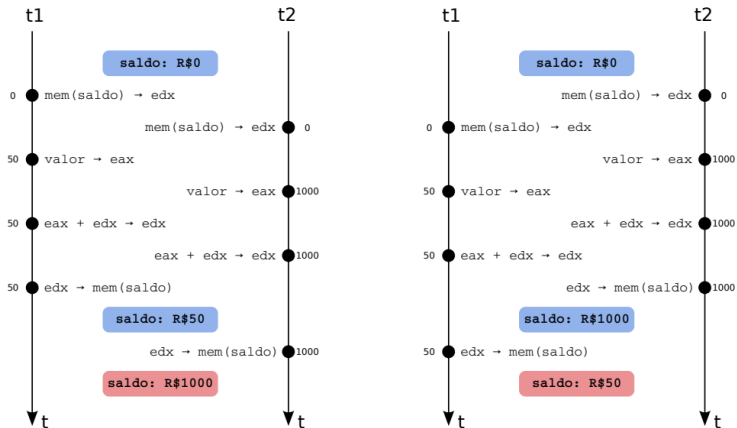
Uma aplicação concorrente



Execuções independentes



Execuções concorrentes



```
#include <stdio.h>
#include <pthread.h>

int saldo = 0; // recurso compartilhado

void* depositar(void* arg) {
    int valor = *(int*)arg;
    int temp = saldo;    // lê o saldo
    temp += valor;       // soma o depósito
    saldo = temp;        // escreve o novo saldo
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int v1 = 50, v2 = 1000;

    pthread_create(&t1, NULL, depositar, &v1);
    pthread_create(&t2, NULL, depositar, &v2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Saldo final: %d\n", saldo);
    return 0;
}
```



Condição de Disputa (Race Condition)

- O que vimos no exemplo da conta se chama **condição de corrida/disputa**.
- Erros gerados por acessos concorrentes a dados
- O resultado final depende do entrelaçamento (timing) das tarefas.
- Se as operações se intercalam, um depósito pode ser "perdido".
- Envolvem ao menos uma operação de escrita
- Podem ocorrer em qualquer sistema concorrente

Quando acontece?

Quando duas ou mais threads ou processos acessam e manipulam um recurso compartilhado (como uma variável na memória) ao mesmo tempo, e o resultado final depende da ordem em que essas operações ocorrem.

Consequência = resultados imprevisíveis ou incorretos.



Condição de Disputa (Race Condition)

Condições de disputa são erros dinâmicos:

- Não aparecem no código fonte
- Só se manifestam durante a execução
- São difíceis de detectar
- Podem ocorrer raramente ou mesmo nunca
- Sua depuração pode ser muito complexa

Definição Formal

São erros que não aparecem no código-fonte e só se manifestam durante a execução, sendo raros e difíceis de depurar.

É melhor prevenir condições de disputa que consertá-las.

Uma condição de disputa é um fantasma que pode assombrar seu código por anos sem nunca ser visto.



Formalizam quando duas tarefas (t_1 e t_2) podem rodar em paralelo sem riscos:

As Regras ($t_1 || t_2$)

- 1 $R(t_1) \cap W(t_2) = \emptyset$ (t_1 não lê o que t_2 escreve)
- 2 $R(t_2) \cap W(t_1) = \emptyset$ (t_2 não lê o que t_1 escreve)
- 3 $W(t_1) \cap W(t_2) = \emptyset$ (ambas não escrevem no mesmo lugar)

Dica de Ouro

Leituras simultâneas não geram disputa. O problema nasce quando há pelo menos uma **escrita**.



- Trechos de código que manipulam dados compartilhados.
- As seções críticas das tarefas t1 e t2 são:

```
(*saldo) += valor;
```

- se várias threads entram na seção crítica ao mesmo tempo, há risco de inconsistência.

```
int temp = saldo;    // lê o saldo
temp += valor;       // soma o depósito
saldo = temp;        // escreve o novo saldo
```

- Se duas threads executarem esse trecho simultaneamente, pode ocorrer uma condição de disputa e o resultado final será incorreto.

Definição

é o trecho do código onde ocorre o acesso a um recurso compartilhado que não pode ser usado por mais de uma thread ao mesmo tempo sem causar problemas.



- Seção crítica é como uma ponte estreita de mão única.
- Só cabe um carro por vez atravessando.
- Se dois carros tentarem entrar juntos, vão bater ou ficar travados.
- A ponte representa o trecho de código que acessa o recurso compartilhado.



Solução para ponte: Colocar um mecanismo de controle de acesso.

Solução para SO: Usar mecanismos de exclusão mútua.



Exclusão Mútua

Propriedade de garantir que apenas uma tarefa esteja em sua seção crítica por vez.

Analogia do Banheiro

A seção crítica é como um banheiro de uso individual: para entrar, você precisa da chave (enter). Enquanto estiver lá dentro, ninguém mais entra. Ao sair, você devolve a chave (leave).

Foco

Impedir o entrelaçamento de seções críticas, de modo que apenas uma tarefa esteja na seção crítica a cada instante.



Exclusão Mútua: A Regra de Ouro

- **O que é:** Uma propriedade que garante que, se uma tarefa está executando em sua seção crítica, nenhuma outra tarefa pode entrar na mesma seção crítica.

Analogia: A Cabine Telefônica

Imagine uma cabine telefônica antiga:

- 1 Apenas uma pessoa cabe lá dentro por vez.
- 2 Enquanto alguém fala ao telefone (Seção Crítica), os outros esperam do lado de fora.
- 3 Somente quando a pessoa sai (Liberação), a próxima pode entrar.

Primitivas de Controle

Para funcionar, o programador deve delimitar o código com:

- **Enter(recurso):** Solicita permissão (bloqueia se ocupado).
- **Leave(recurso):** Avisa que terminou e libera o acesso.



- Para evitar condições de disputa, o acesso às variáveis compartilhadas deve ser protegido.
- O código da operação de depósito pode ser reescrito assim:

```
void depositar (long conta, long *saldo, long valor){  
    enter (conta) ; // entra na seção crítica "conta"  
    (*saldo) += valor ; // usa as variáveis compartilhadas  
    leave (conta) ; // sai da seção crítica  
}
```

- A variável **conta** identifica a seção crítica.
- As funções **enter()** e **leave()** garantem a exclusão mútua:
 - Apenas uma thread pode acessar a seção crítica por vez.
 - Evita inconsistências no recurso compartilhado.



Critérios para Exclusão Mútua

Para que um mecanismo de exclusão mútua seja considerado correto e robusto, ele deve atender a quatro requisitos fundamentais:

Os 4 Pilares

Exclusão Mútua Apenas uma tarefa por vez na seção crítica.

Espera Limitada Nenhuma tarefa deve esperar para sempre (fim da inanição).

Independência Tarefas fora da seção crítica não podem impedir outras de entrar.

Fatores Físicos A solução deve ser lógica (não depender da velocidade da CPU ou número de núcleos).

Uma solução de exclusão mútua que depende da 'sorte' do escalonador não é uma solução, é uma bomba relógio.



- Vamos ver um exemplo que temos o problema de condição de corrida
- Vamos identificar a seção crítica
- contador.c



- Desativar as interrupções ao entrar na seção crítica e reativar ao sair .

Por que é perigoso?

- Se a tarefa travar, o sistema inteiro trava.
- Pode causar perda de dados de rede/disco.
- **Não funciona em Multiprocessadores** (outros núcleos continuam rodando).

Uso restrito ao núcleo (Kernel) do SO.



A solução trivial

- Usamos uma variável global que funciona como uma **flag** indicando se a seção crítica está:
 - Ocupada (`busy = 1`)
 - Livre (`busy = 0`).

```
int busy = 0 ; // a seção está inicialmente livre

void enter (){
    while (busy) ; // espera enquanto a seção estiver ocupada
    busy = 1 ; // marca a seção como ocupada
}

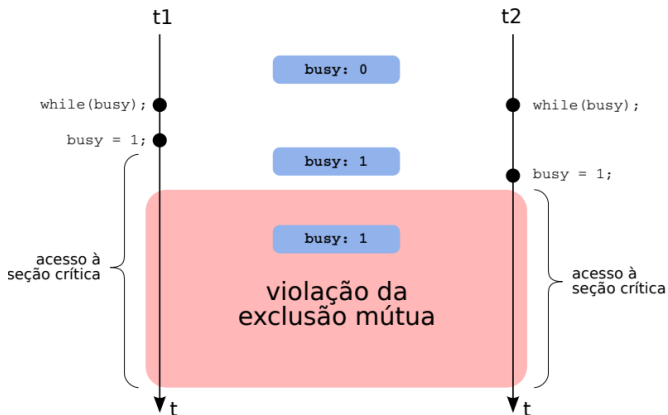
void leave (){
    busy = 0 ; // libera a seção (marca como livre)
}
```

- `enter()` fica em um loop de espera ativa (`busy waiting`) até que a seção esteja livre.
- Quando entra, marca como ocupada.
- `leave()` libera a seção crítica.

Vamos testar!



A solução trivial



O Defeito Fatal

O teste da variável e a atribuição ocorrem em momentos distintos. Uma troca de contexto entre eles permite que duas tarefas leiam "busy=0" e entrem juntas.

- **Espera Ocupada (Busy Wait):** O teste contínuo consome processador inutilmente.
 - a thread fica presa em um loop consumindo CPU enquanto espera, desperdiçando recursos.

Analogia - Banheiro de Bar

- Se duas pessoas chegarem ao mesmo tempo e olharem a plaquinha "livre", ambas podem tentar entrar juntas.
- Além disso, quem espera fica parado olhando a porta (espera Ocupada), em vez de poder fazer outra coisa até o banheiro ficar disponível.



- Existe uma variável global `turn` que indica qual tarefa tem o direito de entrar na seção crítica.
- Cada tarefa só entra quando `turn == task`.
- Ao sair, a tarefa atual passa o turno para a próxima $((turn + 1) \% num_tasks)$.

```
int num_tasks;
int turn = 0; // inicia pela tarefa 0

void enter(int task) // task vale 0, 1, ..., num_tasks-1
{
    while (turn != task) {
        // a tarefa espera seu turno
    }
}

void leave(int task)
{
    turn = (turn + 1) % num_tasks; // passa para a próxima tarefa
}
```



Analogia - Banheiro de Bar

- A variável turn é como uma senha numerada (0, 1, 2, ...).
- Cada pessoa só pode entrar quando sua senha for chamada.
- Ao sair, ela passa a vez para a próxima senha.

Ninguém entra fora de ordem e todos têm sua vez garantida.

Vamos testar!



Problema da Dependência

Se for a vez da Tarefa 0, mas ela não quiser entrar na seção crítica, a Tarefa 1 fica bloqueada para sempre esperando seu turno.

Analogia - Banheiro de Bar

- A Pessoa 1 pegou a senha, mas não está com vontade de usar o banheiro agora.
- A Pessoa 3 já bebeu bastante, está apertada e precisa urgentemente usar o banheiro.
- Porém, como o sistema é rígido, a Pessoa 3 só pode entrar quando chegar a vez da senha 3. Enquanto isso, ela precisa esperar a Pessoa 1, que nem tem necessidade de entrar.



Cada processo sinaliza que deseja entrar na seção crítica e cede a vez ao outro.

Se ambos quiserem entrar, a variável `turn` decide quem entra primeiro.

Progresso: se apenas um quer entrar, ele entra imediatamente.

Justiça: se ambos querem, o `turn` garante alternância, evitando que um monopolize o recurso.



Analogia - Banheiro de Bar

- Dois clientes (Pessoa A e Pessoa B)
- Cada pessoa coloca uma plaquinha “quero entrar” (isso é o flag).
- Além disso, existe uma plaquinha “vez” que aponta para A ou para B (isso é o turn).
- Se os dois querem entrar ao mesmo tempo:
 - A olha a plaquinha “vez”. Se estiver apontando para B, A espera.
 - B olha a plaquinha “vez”. Se estiver apontando para A, B espera.
- Assim, mesmo que os dois estejam apertados, apenas um entra de cada vez, e o outro espera sua vez.
- Quando alguém sai, tira sua plaquinha de “quero entrar” e o banheiro fica livre para o outro.



Algoritmo de Peterson (1981)

- A tarefa expressa o desejo de entrar, mas gentilmente cede o turno para a outra.
- Se a outra não quiser, ela entra imediatamente.

```
int turn = 0;           // indica de quem é a vez
int wants[2] = {0, 0}; // indica se a tarefa i quer acessar a seção crítica

void enter(int task) { // task pode valer 0 ou 1

    int other = 1 - task; // indica a outra tarefa
    wants[task] = 1;      // task quer acessar a seção crítica
    turn = other;
    while ((turn == other) && wants[other]) {
        // espera ocupada
    }
}

void leave(int task) {
    wants[task] = 0; // task libera a seção crítica
}
```

- Uma solução elegante para **duas tarefas** que combina a intenção (wants) e a vez (turn).
- Garante exclusão mútua e espera limitada.



- **Definição:** instruções que o processador executa de forma indivisível — ou acontecem por completo, ou não acontecem.
- **Benefício:** garantem consistência dos dados em ambientes concorrentes (múltiplas threads/processos).
- **Exemplo clássico:** incremento de variável compartilhada.
 - `saldo++` pode ser quebrado em: ler, somar, escrever.
 - Se não for atômico, outra thread pode interferir entre essas etapas.
- **Características:**
 - Usam instruções da CPU para garantir indivisibilidade.
 - São leves e rápidas.
 - Ideais para operações simples (incremento, troca de valor).
 - Para trechos maiores de código, ainda é necessário exclusão mútua.



Operações atômicas - TSL

- A solução definitiva para a variável busy: instruções de máquina que testam e alteram um valor em um único passo indivisível.
- Instruções de máquina específicas para exclusão:
 - **TSL (Test-and-Set Lock)**: Lê o valor antigo e escreve 1 atomicamente.
 - **XCHG (Exchange)**: Troca valores entre registrador e memória.

```
int lock = 0; // variável de trava

void enter(int *lock){ // passa o endereço da trava
    while (TSL(*lock)) {
        // espera ocupada sobre a trava
    }
}

void leave(int *lock) {
    *lock = 0; // libera a seção crítica
}
```

- Comportamento da instrução TSL

$$TSL(x) = \begin{cases} \text{old} \leftarrow x & // \text{guarda o valor anterior} \\ x \leftarrow 1 & // \text{ativa o flag} \\ \text{return}(\text{old}) & // \text{devolve o valor anterior} \end{cases}$$



Imagine que x é um cadeado ($0 =$ aberto, $1 =$ fechado).

- 1 `old <- x`: Você tira uma "foto" do estado atual do cadeado.
- 2 `x <- 1`: Sem soltar a maçaneta, você gira a chave para trancar (independente de já estar trancado ou não, você garante que agora está).
- 3 `return(old)`: Você olha para a "foto" que tirou no passo 1 para saber o que aconteceu.

O processo que chama o TSL interpreta o resultado da seguinte forma:

- Se o retorno for 0: "Opa, a foto mostrou que estava aberto! Como eu acabei de trancar (no passo 2), agora a sala é minha." → Entra na Seção Crítica.
- Se o retorno for 1: "Ih, a foto mostrou que já estava trancado. Alguém já está lá dentro." → Fica em loop (espera ocupada) tentando de novo.



- **Ineficiência:** uso de espera ocupada (teste contínuo de uma condição).
 - As tarefas ficam consumindo CPU sem necessidade.
 - O ideal seria suspender a tarefa até o recurso ficar disponível.
- **Injustiça:** não há garantia de ordem no acesso.
 - Uma tarefa pode entrar e sair várias vezes em sequência.
 - Outras tarefas podem ficar esperando indefinidamente.
- Essas soluções são usadas apenas:
 - Dentro do núcleo do Sistema Operacional.
 - Em sistemas muito simples ou didáticos.

Próximo passo: usar mecanismos mais robustos como *mutexes* e *semáforos*.



Atividade



```
int saldo = 100;
void* depositar(void* arg) {
    int valor = *(int*)arg;
    int temp = saldo;          // Passo 1: Leitura
    // Estou forçando a troca de contexto aqui:
    // A thread pausa por alguns microssegundos,
    // dando chance para a outra thread ler o mesmo saldo original.
    usleep(100);
    temp += valor;             // Passo 2: Soma
    saldo = temp;              // Passo 3: Escrita
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int v1 = 50, v2 = 30;
    pthread_create(&t1, NULL, depositar, &v1);
    pthread_create(&t2, NULL, depositar, &v2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Saldo final esperado: 180\n");
    printf("Saldo final obtido: %d\n", saldo);
    return 0;
}
```

- Aplicar os algoritmos a baixo no problema da conta
 - Alternância de Uso
 - Algoritmo de Peterson
 - TSL

